



OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Architecture Pattern - MVC

Archana A

Department of Computer Applications

The Model-View-Controller architectural pattern (MVC) divides an interactive application into **three components**

- A ***model*** contains the core functionality and data.
- ***Views*** *present data and information to the user.*
- ***Controllers*** *are associated with views and allow manipulation of the presented data and information.*
- A **change-propagation mechanism** ensures consistency between the user interface and the model

Context:

- Interactive applications with a flexible human-computer interface.

Problem:

The following forces influence the solution:

- The same information is presented differently in different windows, for example, in a bar or pie chart.

- The display and behavior of the application must **reflect data manipulations immediately**.
- Changes to the user interface should be easy, and even possible at run-time.
- Supporting different 'look and feel' standards or porting the user interface **should not affect code** in the **core of the application**

Solution:

MVC divides an interactive application into the three areas: **processing**, **output**, and **input**.

- The **model** component encapsulates **core data and functionality**. The model is independent of specific output representations or input behavior.

- **View** components **display information** to the user. A view **obtains the data from the model**. There can be multiple views of the model.
- Each view has an associated controller component.
- **Controllers** receive input, usually as **events** that encode mouse movement, activation of mouse buttons, or keyboard input.
- Events are translated to service requests for the model or the view. The user interacts with the system solely through controllers.

Structure:

- MVC design pattern contains three components:
 - Model
 - View
 - Controller

Class Model	Collaborators • View • Controller
Responsibility • Provides functional core of the application. • Registers dependent views and controllers. • Notifies dependent components about data changes.	

View: **View** components display information to the user.

Class View	Collaborators • Controller • Model
Responsibility • Creates and initializes its associated controller. • Displays information to the user. • Implements the update procedure. • Retrieves data from the model.	

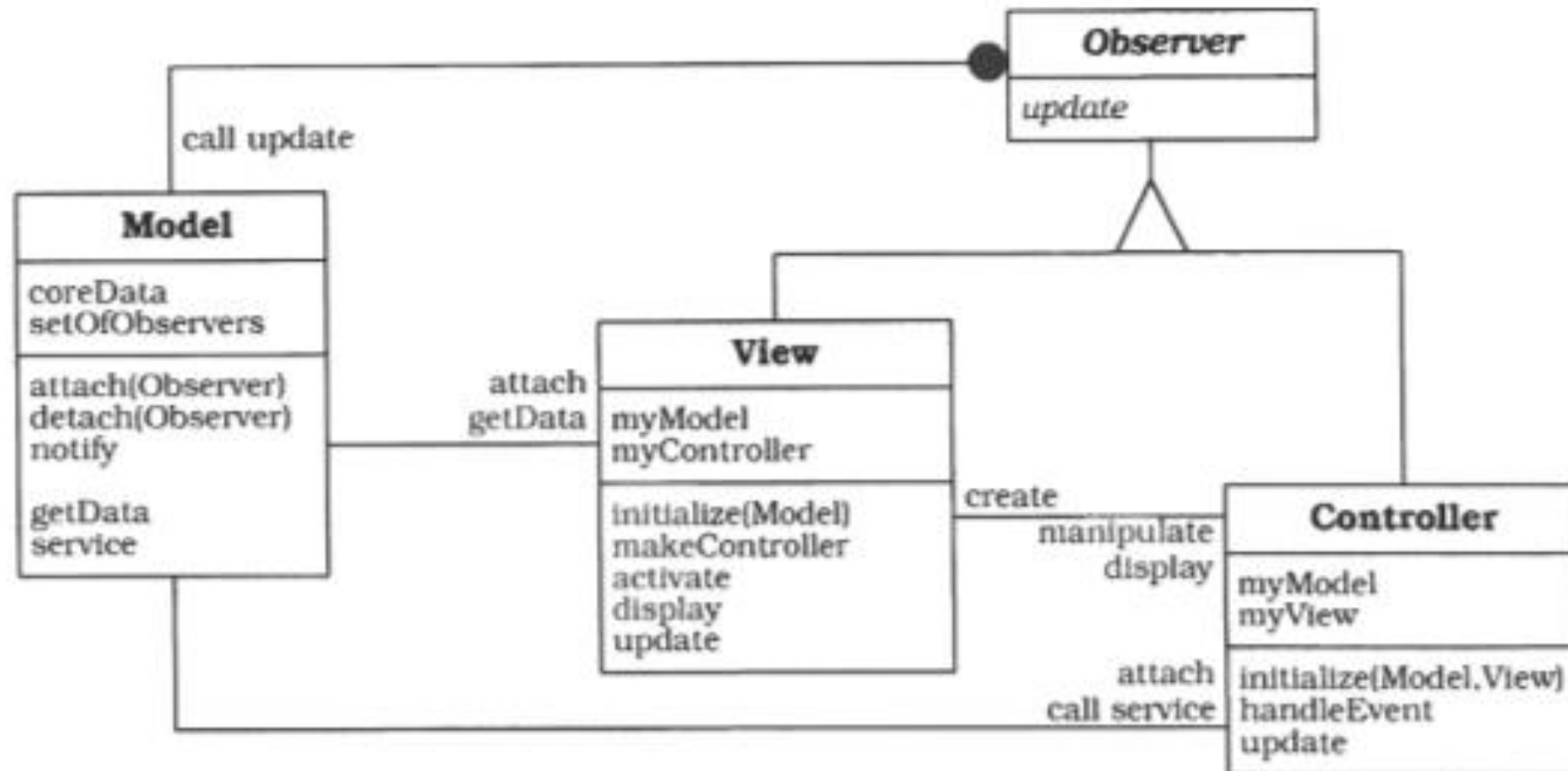
Controller: these *are associated with views and allow manipulation of the presented data and information*

Class Controller	Collaborators • View • Model
Responsibility • Accepts user input as events. • Translates events to service requests for the model or display requests for the view. • Implements the update procedure, if required.	

OBJECT ORIENTED SOFTWARE ENGINEERING

MVC Pattern - General Structure:

The **general structure** of MVC design pattern is shown below:



Variants:

Document-View: This variant relaxes the separation of view and controller. In several GUI platforms, window display and event handling are closely interwoven.

Known Uses:

Smalltalk [GR83]. The best-known example of the use of the Model- View-Controller pattern is the user-interface framework in the Smalltalk environment .

MVC was established to build reusable components for the user interface.

.

Consequences:

Benefits:

- Multiple views of the same model.

- Synchronized views

- Exchangeability of 'look and feel'.

Liabilities:

- Increased complexity

- Potential for excessive number of updates

- Intimate connection between view and controller

- Close coupling of views and controllers to a model. Both view and controller components make direct calls to the model

Model (ShapeModel):

- Manages the state of the shape (color).
- Provides methods to change and retrieve the color.

View (ShapeView):

- Displays the shape (rectangle) and a button.
- Updates the UI when the color changes.

Controller (ShapeController):

- Handles the button click event.
- Updates the Model and notifies the View to reflect the changes.

Main Application:

- Initializes the Tkinter window and starts the MVC-based application.

MVC Application example Working:

- When the "Change Color" button is clicked:
 - The **Controller** calls the `change_color` method in the **Model**.
 - The **Model** updates the color.
 - The **Controller** retrieves the new color and updates the **View**.
 - The **View** changes the color of the rectangle on the canvas.

MVC pattern – example2:

E-commerce website:

Imagine a product page on an e-commerce site.

- The **Model** would represent the **product data** (description, price, inventory).
- The **View** would be the **product page layout** displaying the product information with images.
- The **Controller** would handle **user actions** like **adding the product to the cart**, which would update the model (inventory) and trigger a view update (cart indicator).

MVC Application – example3:

Social media platform:

- In a social media feed, the **Model** would be the posts, user profiles, and comments stored in the database.
- The **View** would be the user interface showing the feed content, profiles, and comment sections.
- The **Controller** would handle user interactions like posting updates, liking comments, which would modify the model (create new posts, update like count) and cause the view to reflect the changes (new post appears in feed, like count updates).



THANK YOU

Ms. Archana A

Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392